



The 'this' Keyword

The Most Confusing Part of JavaScript — Explained

Covering: 6 Rules, Arrow vs Regular, Event Listeners, Classes, call/apply/bind, React Patterns, and a Complete Cheat Sheet

Prepared by Claude for Akshay

Table of Contents

1. What IS 'this'?
2. Rule 1 — this in Object Methods
3. Rule 2 — this Alone (No Object)
4. Rule 3 — this in Arrow Functions (The Big One)
5. Rule 4 — this in Event Listeners
6. Rule 5 — this in Classes & React
7. Rule 6 — Explicit Binding (call, apply, bind)
8. Tricky Example — Full Walkthrough
9. The Complete Cheat Sheet
10. 5 Golden Rules to Remember

1. What IS 'this'?

`this` is a special keyword that refers to **the object that is currently calling the function**. It's NOT fixed — it changes depending on **HOW** the function is called.

■ The Analogy

Think of `this` like the word "**me**" — when Akshay says "me", it refers to Akshay. When Sara says "me", it refers to Sara. Same word, different person based on **who is speaking**.

The value of `this` is determined at **call time**, not at definition time. This is what makes it confusing — the same function can have different `this` values depending on how it's invoked.

2. Rule 1 — this in Object Methods

this = the object that owns the method

```
const akshay = {
  name: "Akshay",
  greet() {
    console.log(this);      // the whole akshay object
    console.log(this.name); // "Akshay"
  }
};
akshay.greet(); // "Akshay"
// WHO called greet? -> akshay did -> this = akshay
```

```
const sara = {
  name: "Sara",
  greet() {
    console.log(this.name);
  }
};
sara.greet(); // "Sara"
// WHO called greet? -> sara did -> this = sara
```

✓ The Dot Rule

Look **LEFT of the dot**. That's this.

akshay.greet() — left of dot = akshay — this = akshay
sara.greet() — left of dot = sara — this = sara

What happens when you extract the method?

```
const akshay = {
  name: "Akshay",
  greet() {
    console.log(this.name);
  }
};
// Calling on the object - works
akshay.greet(); // "Akshay"
// Extracting the function - BREAKS!
const fn = akshay.greet;
fn();           // undefined!
```

```
// fn() has nothing left of the dot!
```

■ Method Extraction Trap

When you pull a method out of its object and call it alone, `this` is lost. This happens often when passing methods as callbacks: `setTimeout(user.greet, 1000)` will lose `this`!

3. Rule 2 — this Alone (No Object)

In browser: this = window. In strict mode: this = undefined.

```
function showThis() {  
  console.log(this);  
}  
showThis();  
// In browser:      window object  
// In strict mode:  undefined  
// In Node.js:      global object
```

There's **nothing left of the dot**, so this falls back to the default. Since most modern code uses strict mode (modules, classes, and React all default to strict mode), you'll usually get undefined.

```
"use strict";  
function showThis() {  
  console.log(this); // undefined  
}  
showThis();
```

4. Rule 3 — this in Arrow Functions

Arrow functions do NOT have their own `this`. They STEAL it from their parent scope. This is the most important rule.

Arrow as Object Method — BREAKS!

```
const user = {
  name: "Akshay",
  // Regular function -> has its OWN this
  regularGreet: function() {
    console.log(this.name); // "Akshay"
  },
  // Arrow function -> NO own this, looks OUTSIDE
  arrowGreet: () => {
    console.log(this.name); // undefined!
    // 'this' here = whatever 'this' is OUTSIDE user
    // which is window/undefined, NOT user!
  }
};
user.regularGreet(); // "Akshay"
user.arrowGreet();  // undefined
```

■ Never Use Arrow as Object Method

Arrow functions don't have their own `this`, so `this.name` inside an arrow method won't refer to the object. Always use regular functions or method shorthand for object methods.

Arrow Inside a Method — THE FIX!

This is the **main reason arrow functions were invented**:

```
const user = {
  name: "Akshay",
  friends: ["Sara", "John", "Mike"],
  showFriends: function() {
    console.log(this.name); // "Akshay"
    // PROBLEM: regular function callback LOSES this
    this.friends.forEach(function(friend) {
      console.log(`${this.name} knows ${friend}`);
      //           this.name = undefined! BROKEN!
    });
  }
}
```

```
};
user.showFriends();
// "Akshay"
// "undefined knows Sara"
// "undefined knows John"
// "undefined knows Mike"
```

WHY did this break? Because `forEach` calls the inner function as a plain function (no `object.dot`), so `this` becomes `undefined`.

The Fix — Arrow Function:

```
const user = {
  name: "Akshay",
  friends: ["Sara", "John", "Mike"],
  showFriends: function() {
    console.log(this.name); // "Akshay"
    // Arrow INHERITS this from showFriends
    this.friends.forEach((friend) => {
      console.log(`${this.name} knows ${friend}`);
      //      this.name = "Akshay"! WORKS!
    });
  }
};
user.showFriends();
// "Akshay"
// "Akshay knows Sara"
// "Akshay knows John"
// "Akshay knows Mike"
```

✓ The Arrow Function Rule

Use arrows INSIDE methods (to keep this alive in callbacks). **Don't use arrows AS methods** (they'll break this). This is the single most important thing to remember about arrow functions.

5. Rule 4 — this in Event Listeners

```
const button = document.getElementById("myBtn");
// Regular function -> this = the element that fired the event
button.addEventListener("click", function() {
  console.log(this);           // <button> element
  this.style.color = "red";    // works!
});
// Arrow function -> this = inherited from outside
button.addEventListener("click", () => {
  console.log(this);           // window (not the button!)
  this.style.color = "red";    // BREAKS!
});
```

■ When to Use Which in Event Listeners

Need to access the element (this)? Use a regular function.

Need to access outer scope variables? Use an arrow function.

Modern approach: Use arrow function + `event.target` or `event.currentTarget` instead of `this`.

```
// Modern approach - avoid 'this' entirely
button.addEventListener("click", (event) => {
  console.log(event.target);           // <button> element
  event.target.style.color = "red";    // works!
  // No confusion about 'this' at all
});
```

6. Rule 5 — this in Classes & React

this in Classes

```
class User {
  constructor(name) {
    this.name = name; // this = the new object being created
  }
  greet() {
    return `Hi, I'm ${this.name}`;
  }
}
const akshay = new User("Akshay");
akshay.greet(); // "Hi, I'm Akshay"
// 'new' keyword makes this = the new object
```

Classic React Class Component Bug

```
class MyComponent extends React.Component {
  constructor() {
    super();
    this.state = { count: 0 };
  }
  // BUG: 'this' is lost when passed as event handler
  handleClick() {
    this.setState({ count: this.state.count + 1 });
    // TypeError: Cannot read 'setState' of undefined
  }
  render() {
    // When React calls handleClick, 'this' is undefined
    return <button onClick={this.handleClick}>Click</button>;
  }
}
```

■ Why This Breaks

When you pass `this.handleClick` to `onClick`, React stores the function reference and calls it later WITHOUT the object. It's the same as extracting a method: `const fn = obj.method;` `fn();` — `this` is lost!

Three Fixes

```
// Fix 1: Bind in constructor (old way)
constructor() {
  super();
  this.handleClick = this.handleClick.bind(this);
}

// Fix 2: Arrow function in property (common way)
handleClick = () => {
  this.setState({ count: this.state.count + 1 });
  // Arrow inherits 'this' from the class -> works!
}

// Fix 3: Arrow in JSX (works but creates new fn each render)
<button onClick={() => this.handleClick()}>Click</button>
```

Modern React — No 'this' At All!

This is exactly why modern React uses **functional components with hooks**:

```
// Modern React - no 'this' needed anywhere!
const MyComponent = () => {
  const [count, setCount] = useState(0);
  const handleClick = () => {
    setCount(count + 1); // no 'this' anywhere!
  };
  return <button onClick={handleClick}>Click {count}</button>;
};
```

✓ Modern React = No 'this' Problems

Functional components + hooks completely eliminate this confusion. This is one of the biggest reasons the React community moved away from class components. If you're writing new React code, you'll rarely need to think about this.

7. Rule 6 — Explicit Binding (call, apply, bind)

You can **manually** set what this should be:

call() — Call Immediately With Given 'this'

```
function greet() {
  return `Hi, I'm ${this.name}`;
}
const akshay = { name: "Akshay" };
const sara = { name: "Sara" };
greet.call(akshay);    // "Hi, I'm Akshay"
greet.call(sara);     // "Hi, I'm Sara"
```

call() vs apply() — Passing Arguments

```
function intro(greeting, emoji) {
  return `${greeting}, I'm ${this.name} ${emoji}`;
}
const akshay = { name: "Akshay" };
// call - args one by one
intro.call(akshay, "Hey", "!");
// "Hey, I'm Akshay !"
// apply - args as array
intro.apply(akshay, ["Hey", "!"]);
// "Hey, I'm Akshay !"
```

■ Memory Trick

call = **C**ommas (args separated by commas)

apply = **A**rray (args as an array)

bind() — Returns a New Function With 'this' Locked

```
function greet() {
  return `Hi, I'm ${this.name}`;
}
const akshay = { name: "Akshay" };
// bind returns a NEW function with 'this' permanently locked
const akshayGreet = greet.bind(akshay);
akshayGreet();    // "Hi, I'm Akshay"
```

```
// Always Akshay, no matter how you call it
// Even if you try to override:
const sara = { name: "Sara" };
akshayGreet.call(sara); // STILL "Hi, I'm Akshay"
// bind is permanent!
```

Real-World bind() Use Case

```
const user = {
  name: "Akshay",
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};
// Without bind - this is lost!
setTimeout(user.greet, 1000);
// "Hi, I'm undefined"
// With bind - this is locked!
setTimeout(user.greet.bind(user), 1000);
// "Hi, I'm Akshay"
// Or just use an arrow function (modern way):
setTimeout(() => user.greet(), 1000);
// "Hi, I'm Akshay"
```

Method	When It Runs	How to Pass Args
call(thisArg, a, b)	Immediately	One by one (commas)
apply(thisArg, [a, b])	Immediately	As array
bind(thisArg)	Returns new function	When calling later

8. Tricky Example — Full Walkthrough

Let's trace through a complete example step by step:

```
const restaurant = {
  name: "Desi Bites",
  menu: ["Biryani", "Paneer", "Dosa"],
  showMenu: function() {
    console.log(`Welcome to ${this.name}`);
    // this = restaurant (restaurant.showMenu())
    this.menu.forEach((item) => {
      console.log(`${this.name} serves ${item}`);
      // this = restaurant (arrow inherits from showMenu)
    });
  }
};
```

Calling on the object:

```
restaurant.showMenu();
// Output:
// "Welcome to Desi Bites"
// "Desi Bites serves Biryani"
// "Desi Bites serves Paneer"
// "Desi Bites serves Dosa"
// WHY: restaurant is left of dot -> this = restaurant
// Arrow function inherits this from showMenu -> still restaurant
```

Extracting the method:

```
const fn = restaurant.showMenu;
fn();
// Output:
// "Welcome to undefined"
// TypeError: Cannot read 'forEach' of undefined
// WHY: fn() has nothing left of the dot
// this = undefined (strict mode)
// this.menu = undefined -> forEach crashes!
```

Fixing with bind:

```
const fn = restaurant.showMenu.bind(restaurant);
fn();
// Output:
```

```
// "Welcome to Desi Bites"
// "Desi Bites serves Biryani"
// "Desi Bites serves Paneer"
// "Desi Bites serves Dosa"
// bind locked 'this' to restaurant permanently
```

Another Tricky One — Method Sharing

```
const akshay = {
  name: "Akshay",
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};

const sara = { name: "Sara" };
// "Borrow" akshay's method for sara
sara.greet = akshay.greet;
akshay.greet(); // "Hi, I'm Akshay"
sara.greet();   // "Hi, I'm Sara"
// Both call the SAME function, but 'this' differs!
// akshay.greet() -> left of dot = akshay
// sara.greet()   -> left of dot = sara
```

✓ Key Takeaway

this is determined by HOW you CALL the function, not where you DEFINE it. The same function can have completely different this values depending on what's left of the dot when it's invoked.

9. The Complete Cheat Sheet

How Function is Called	this Equals	Example
object.method()	The object	user.greet() -> this = user
Plain function fn()	window / undefined	greet() -> this = window
Arrow function	Inherited from parent	Same this as surrounding code
new Constructor()	The new object	new User() -> this = new obj
fn.call(obj)	Whatever you pass	greet.call(sara) -> this = sara
fn.apply(obj, [])	Whatever you pass	greet.apply(sara) -> this = sara
fn.bind(obj)()	Locked to obj	Always this = obj
Event listener (regular)	The DOM element	this = button
Event listener (arrow)	Inherited (usually window)	NOT the element

Priority Order (When Rules Conflict)

If multiple rules could apply, JavaScript follows this priority:

```
// Priority (highest to lowest):  
// 1. new keyword      -> this = new object  
// 2. call / apply / bind -> this = specified object  
// 3. object.method()  -> this = the object  
// 4. plain function() -> this = window / undefined  
// Arrow functions IGNORE all of the above  
// They ALWAYS inherit from parent scope
```


10. 5 Golden Rules to Remember

Rule 1: Look Left of the Dot

Whatever is to the left of `.method()` is `this`. `user.greet()` means `this = user`.

Rule 2: No Dot = No this

If a function is called without an object (`greet()`), `this` falls back to `window` or `undefined` (strict mode).

Rule 3: Arrow Functions Steal this

Arrow functions never have their own `this`. They grab it from their surrounding scope. Use them INSIDE methods, not AS methods.

Rule 4: Use Arrows Inside, Regular Outside

Use arrow functions inside methods (callbacks, `map`, `forEach`) to keep `this` alive. Use regular functions or method shorthand as the method itself.

Rule 5: Modern React Avoids All This

Functional components + hooks = no `this` at all. If you're writing new React code, `this` is rarely relevant.

You now understand the most confusing concept in JavaScript! The good news is — with modern React and functional components, you'll rarely need to worry about `this` in your daily work. But when you do encounter it (interviews, legacy code, vanilla JS), you'll know exactly what's happening.